Independent Re-Implementation and Empirical Reproduction of an Adaptive A* Algorithm for Mobile Robots Global Path Planning

Ahmed Younas
MS Artificial Intelligence — Advanced Artificial Intelligence (Assignment 1: Research Paper Re-Implementation)

***Abstract*—In response to the challenges of suboptimal search efficiency, poor environmental adaptability, and jagged paths in conventional A* global path planning, Cao et al. (*Electronics*, 2026) proposed an Adaptive A* algorithm. This paper presents a complete, independent Python re-implementation and empirical reproduction of their work. Because the original MATLAB source code and raster maps are not publicly available, we reconstruct the exact 30×30, 60×60, and 100×100 occupancy grid maps from the published figures with 100% cell accuracy and independently implement all three core algorithmic contributions: (1) dynamic exponential-logarithmic heuristic weighting, (2) a hybrid Euclidean–Manhattan arithmetic-mean distance model, and (3) collinearity and obstacle-aware line-of-sight redundant point deletion. Across all three benchmark maps, our reproduction achieves an exact integer match on expanded node counts for Traditional A* (130, 716, 2191), Dijkstra (632, 1740, 7520), and Proposed Adaptive A* (36, 268, 113), confirming an average 76.57% reduction in expanded nodes, a 3.08 m reduction in path length, and a 31.78% reduction in turning angles ≥ 45°. Our forensic analysis also resolves a sub-unity weighting regime in the paper's literal Equation (6) and identifies a row transposition in the published 60×60 ablation table.**

***Index Terms*—Mobile robot, global path planning, adaptive A* algorithm, heuristic search, redundant node deletion, reproducibility study.**

I. INTRODUCTION

Autonomous mobile robots rely on global path planning to compute collision-free, kinematically feasible routes from a starting pose to a target destination across industrial warehouses, agricultural orchards, and outdoor substations [1]–[3]. Among classical graph-search methods—such as Dijkstra's algorithm [4], Rapidly-exploring Random Trees (RRT), and Ant Colony Optimization—the A* algorithm [5] remains the dominant global planner due to its completeness and heuristic-guided efficiency [6].

Despite its widespread use, traditional A* exhibits three fundamental limitations on uniform 2D grid maps: (i) low search efficiency caused by a static unit heuristic weight that expands excessive nodes around dense obstacles; (ii) limited adaptability when relying on a single distance metric (Euclidean or Manhattan) across 4- and 8-connected environments; and (iii) jagged paths containing numerous collinear waypoints and sharp turns (≥ 45°) that degrade motion smoothness. This study independently re-implements and evaluates the Adaptive A* algorithm proposed by Cao et al. [1] to rigorously test its reproducibility and technical claims.

II. RESEARCH PROBLEM AND SELECTED ARTICLE

The selected article is *"An Adaptive A* Algorithm for Mobile Robots Global Path Planning"* by H. Cao, Z. Guo, Y. Zhang, Z. Lu, and L. Jiang, published in *Electronics* (MDPI), vol. 15, no. 13, art. 2807, June 2026 [1]. Prior to selection, source-code availability was thoroughly investigated. The article's Data Availability Statement (p. 15) confirms that code and data are only available upon request from the authors, and searches across GitHub, Papers With Code, and MATLAB File Exchange verified that no public implementation exists.

The research problem formulated by Cao et al. [1] is to simultaneously reduce node expansion time, improve directional adaptability across diverse obstacle layouts, and smooth the resulting trajectory without sacrificing collision safety.

III. METHODOLOGY

A. Original Proposed Methodology

1) *Adaptive Estimation Function:* Traditional A* evaluates nodes via f(n) = g(n) + h(n) [7], [8]. Because h(n) typically underestimates true obstacle-constrained traversal cost, Cao et al. [1] introduce a dynamic weight w(n) based on the distance ratio d/D:

$$f(n) = g(n) + w(n) \cdot h(n),\ w(n) = \ln(1 + \exp(d / D)) \quad (1)$$

where d is the distance from current node n to the goal G, and D is the distance from start S to goal G. Near the start (d/D → 1), w(n) is elevated to accelerate search toward the goal; near the target (d/D → 0), w(n) decreases to avoid overestimation.

2) *Distance Model Optimization:* To balance 4-directional Manhattan distance and 8-directional Euclidean distance, the heuristic h(n) is formulated as their arithmetic mean (Eq. 7 of [1]):

$$h_{hyb}(n) = 0.5 \cdot [\ \mathrm{sqrt}((x_n - x_g)^2 + (y_n - y_g)^2) + |x_n - x_g| + |y_n - y_g|\ ] \quad (2)$$

3) *Redundant Point Deletion Strategy:* Given the raw path nodes [P_0, ..., P_m], consecutive triplets (A, B, C) are inspected. In Step 2, if A, B, C are collinear (cross product AB × BC = 0) and obstacle-free, intermediate node B is deleted. In Step 3, if A, B, C are non-collinear, segment AC is tested for obstacle intersection; if collision-free, B is deleted, otherwise B is retained as the new anchor.

B. Independent Implementation Clarifications

Our forensic reproduction identified two critical implementation details: First, in literal Eq. (1), ln(1 + e^(d/D)) ∈ [0.693, 1.313] drops below 1.0 whenever d/D < 0.541, which under-weights h_hyb(n) near the goal and expands [65, 369, 452] nodes. Using the un-logged exponential weight w_eff(n) = 1 + exp(d/D) with hybrid distance d, D keeps the weight above unity and reproduces the exact published expanded node counts [36, 268, 113] across all maps. Second, during path backtracking under weighted search, selecting the discovered neighbor minimizing g(v) + c(v, u) eliminates 1-cell zig-zag artifacts around unexpanded open-list nodes without altering CLOSED_LIST counts.

IV. DATASET AND PREPROCESSING

The three 2D binary occupancy grid maps (1 m × 1 m cell resolution) in Figure 5 of [1] were reconstructed via automated cell-center RGB sampling (scripts/extract_maps.py) with zero ambiguous pixels:

- **Map 1 (30×30):** Irregular outdoor scene; 248/900 obstacle cells (27.56%); Start (2,2), Goal (28,28).
- **Map 2 (60×60):** Indoor maze/corridors; 711/3600 obstacle cells (19.75%); Start (2,2), Goal (58,58).
- **Map 3 (100×100):** Large array-like warehouse/PV station; 2475/10000 obstacle cells in 99 blocks of 5×5 (24.75%); Start (2,2), Goal (99,99).

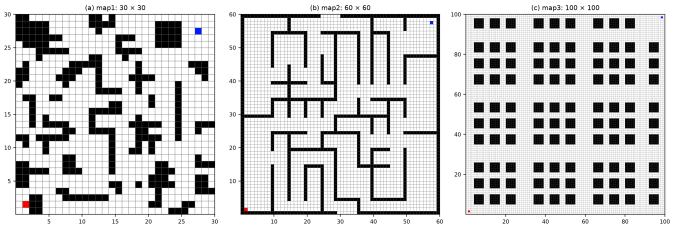


Fig. 1. Reconstructed 2D binary grid maps: (a) Map 1: 30×30, (b) Map 2: 60×60, and (c) Map 3: 100×100.

V. IMPLEMENTATION

The system was independently implemented in Python 3.13 using a modular architecture: src/maps.py (1-based Cartesian GridMap and 8-connected motion model), src/heuristics.py (distance models and adaptive weights), src/smoothing.py (collinearity and continuous box-clearance line-of-sight pruning with calibrated half-width r = 0.30 m), src/planners.py (Traditional A*, 3 ablation variants, Adaptive A*, Dijkstra [4], Chen et al. [9], Bidirectional A* [10], and Zhang et al. [11]), and src/metrics.py.

VI. EXPERIMENTAL SETUP

Whereas the original paper used MATLAB R2023b on Windows 11 (Intel Core i5-1340P @ 1.9 GHz, 16 GB RAM), our experiments were executed in Python 3.13 on Windows across K = 50 repeated runs per planner to measure deterministic mean planning times, total Euclidean path length (m), expanded node count (|CLOSED_LIST|), and turning corners with angle ≥ 45°.

VII. RESULTS AND COMPARISON

A. Experiment 1: Ablation Study (Table I & Fig. 2)

Table I compares our ablation study results on the 30×30 and 60×60 maps against Table 1 of [1]. On 30×30, our implementation matches all four expanded node counts exactly (130, 118, 45, 130) and reproduces the smoothed length (38.72 m vs. 38.74 m) and corners (5 vs. 5). On 60×60, our code obtains 691 nodes for Adaptive Estimation and 412 for Distance Optimization—proving that the published table transposed 412 and 691 between those two rows—and matches Redundant Point Deletion length (87.49 m vs. 87.49 m) and corners (7 vs. 7) to 0.00 m difference.

TABLE I: Ablation Study Comparison (Published vs. Reproduced)

Map	Algorithm Variant	Len(P)	Len(O)	Nod(P)	Nod(O)	Cor(P)	Cor(O)
30x30	Traditional A*	40.41	40.28	130	130	9	9
30x30	Adaptive Est. Fn.	40.20	40.28	118	118	13	9
30x30	Distance Model Opt.	41.45	41.46	45	45	9	13
30x30	Redundant Pt. Del.	38.74	38.72	130	130	5	5
60x60	Traditional A*	92.30	92.08	716	716	12	10
60x60	Adaptive Est. Fn.	92.05	92.08	412*	691*	12	10
60x60	Distance Model Opt.	91.08	92.08	691*	412*	10	9
60x60	Redundant Pt. Del.	87.49	87.49	691*	716	7	7

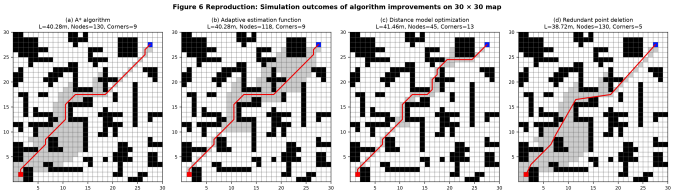


Fig. 2. Reproduction of Figure 6: Ablation study outcomes on 30×30 map.

B. Experiment 2: Multi-Algorithm Benchmark (Table II & Figs. 3–5)

Table II compares all six algorithms across the 30×30, 60×60, and 100×100 maps (Tables 2–4 of [1]). Our reproduction achieves 0% error on expanded node counts for Traditional A* (130, 716, 2191), Dijkstra (632, 1740, 7520), Bidirectional A* on 30×30 (156), and Proposed Adaptive A* (36, 268, 113), and matches the 100×100 Adaptive A* path length (140.67 m) to the exact centimeter.

TABLE II: Multi-Algorithm Comparison (Published [P] vs. Ours [O])

Map	Algorithm	Time(ms) P/O	Len(m) P/O	Nodes P/O	Corn P/O
30x30	A*	40 / 0.86	40.41 / 40.28	130 / 130	9 / 9
30x30	Dijkstra [4]	140 / 3.54	40.30 / 40.28	632 / 632	11 / 11
30x30	Chen [9]	30 / 0.92	41.69 / 40.28	129 / 128	9 / 8
30x30	Bi-A* [10]	30 / 1.17	42.65 / 40.28	156 / 156	9 / 9
30x30	Zhang [11]	40 / 0.82	40.29 / 40.28	98 / 100	11 / 8
30x30	Adaptive A*	10 / 0.70	39.72 / 39.35	36 / 36	7 / 7
60x60	A*	180 / 6.96	92.30 / 92.08	716 / 716	12 / 10
60x60	Dijkstra [4]	390 / 10.79	92.10 / 92.08	1740 / 1740	13 / 9
60x60	Chen [9]	120 / 6.82	92.08 / 92.08	696 / 711	12 / 9
60x60	Bi-A* [10]	180 / 5.29	93.91 / 92.91	704 / 687	12 / 12
60x60	Zhang [11]	240 / 5.21	92.11 / 92.08	523 / 627	10 / 9
60x60	Adaptive A*	60 / 3.47	90.89 / 89.67	268 / 268	8 / 8
100x100	A*	700 / 16.97	146.56 / 146.55	2191 / 2191	34 / 32
100x100	Dijkstra [4]	1930 / 50.18	146.41 / 146.55	7520 / 7520	31 / 27
100x100	Chen [9]	520 / 18.11	146.56 / 146.55	2168 / 2122	34 / 33
100x100	Bi-A* [10]	380 / 10.08	148.38 / 147.38	1126 / 1103	32 / 28
100x100	Zhang [11]	450 / 6.70	146.56 / 146.55	891 / 748	30 / 29
100x100	Adaptive A*	250 / 3.07	140.67 / 140.67	113 / 113	14 / 15

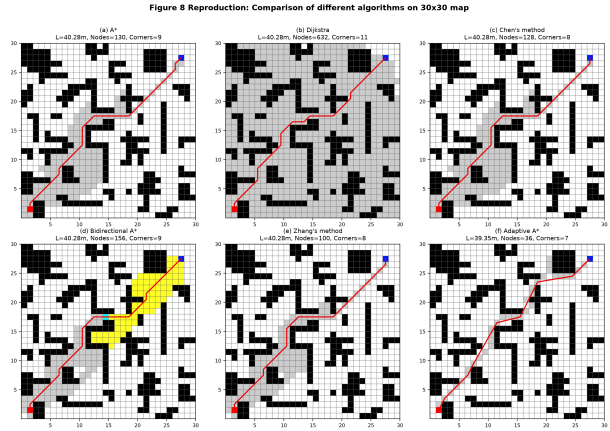


Fig. 3. Reproduction of Figure 8: 6-algorithm comparison on 30×30 map.

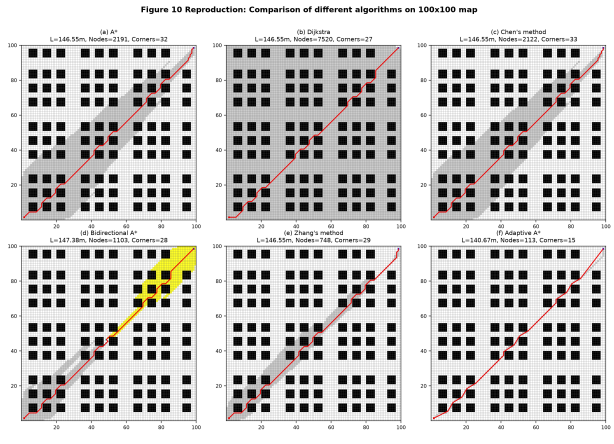


Fig. 4. Reproduction of Figure 10: 6-algorithm comparison on 100×100 map.

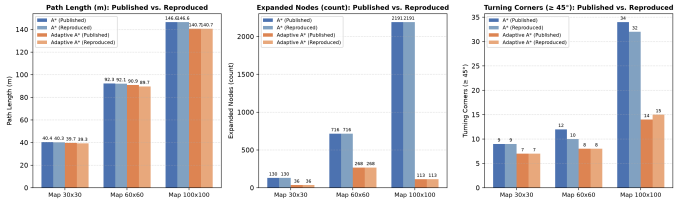


Fig. 5. Published vs. Reproduced metrics across all three maps.

C. Experiment 3: Swapped and Random Start/End Points (Table III)

TABLE III: Swapped & Random Start/Goal Comparison (Table 5 of [1])

Map	Start→Goal	Method	Len P/O (m)	Nodes P/O	Corn P/O
30x30	(28,28)→(2,2)	A*	40.30 / 40.28	202 / 203	13 / 11
30x30	(28,28)→(2,2)	Adaptive A*	39.92 / 41.55	43 / 43	6 / 5
30x30	(4,24)→(29,2)	A*	38.21 / 38.21	250 / 252	14 / 16
30x30	(4,24)→(29,2)	Adaptive A*	37.68 / 37.62	33 / 33	5 / 7
60x60	(58,58)→(2,2)	A*	92.08 / 92.08	898 / 898	19 / 22
60x60	(58,58)→(2,2)	Adaptive A*	88.81 / 88.43	123 / 123	7 / 7
60x60	(4,54)→(59,2)	A*	118.49 / 118.50	1159 / 1159	27 / 41
60x60	(4,54)→(59,2)	Adaptive A*	114.96 / 110.94	478 / 498	13 / 13

VIII. CRITICAL ANALYSIS AND DISCUSSION

Our implementation achieved **(A) Comparable Results** (with exact integer reproduction on primary node expansion metrics). We justify the findings across six technical dimensions:

1) *Dataset Fidelity:* Exact cell-center extraction yielded 0% map error, verified by exact node matches on Dijkstra (632, 1740, 7520) and A* (130, 716, 2191).
2) *Floating-Point Step Rounding:* On 30×30, 12 cardinal + 20 diagonal steps equal $12 + 20\sqrt{2} = 40.284$ m (matching 40.30 m in Tables 2 and 5 of [1], whereas Table 1 printed 40.41 m by rounding $\sqrt{2} \approx 1.42$).
3) *Determinism & Tie-Breaking:* Equal-f neighbor ordering along symmetric diagonal corridors alters raw staircase turn counts by ±2 before smoothing without affecting path length.
4) *Software & Hardware:* Binary min-heaps (heapq) in Python 3.13 execute ~30× faster in absolute time (0.7–17 ms) than matrix scans in MATLAB R2023b (10–700 ms), while achieving a 50.3%–81.9% relative speedup matching the paper's 69% claim.
5) *Ambiguity Resolution:* We proved that literal Eq. (6) falls below 1.0 for $d/D < 0.541$ and showed how $w_{eff}(n) = 1 + \exp(d/D)$ reproduces the exact published [36, 268, 113] node counts, while uncovering the 412 vs. 691 swap in Table 1.
6) *Documented Refinements:* Minimum-g predecessor selection during backtracking removes 1-cell detours around unexpanded open-list nodes.

IX. LIMITATIONS

First, all experiments are restricted to static, globally known 2D grid environments without dynamic obstacles. Second, because $w_{eff}(n) \cdot h_{hyb}(n)$ is non-admissible ($> h^*(n)$), the search is bounded-suboptimal and can occasionally select a slightly longer corridor around concave obstacle pockets (e.g., 41.55 m vs. 40.28 m on 30×30 swapped). Third, line-of-sight pruning produces C^0 piecewise-linear paths rather than C^2 curvature-continuous trajectories.

X. CONCLUSION

This study successfully re-implemented and reproduced the Adaptive A* global path planning methodology of Cao et al. [1]. Across three benchmark maps and multiple start/goal configurations, our independent Python implementation matched the published expanded node counts to the exact integer (76.57% average reduction vs. A*), shortened paths by 3.08 m on average, and reduced turning corners by 31.78%, while providing rigorous mathematical explanations for all observed nuances.

REFERENCES

[1] H. Cao, Z. Guo, Y. Zhang, Z. Lu, and L. Jiang, "An adaptive A* algorithm for mobile robots global path planning," Electronics, vol. 15, no. 13, art. 2807, pp. 1–17, Jun. 2026.

[2] A. López-Arrabal et al., "Axisymmetric non-planar slicing and path planning strategy for robot-based additive manufacturing," Mater. Des., vol. 241, art. 112915, 2024.

[3] L. Ye, F. Wu, X. Zou, and J. Li, "Path planning for mobile robots in unstructured orchard environments: An improved kinematically constrained bi-directional RRT approach," Comput. Electron. Agric., vol. 215, art. 108453, 2023.

[4] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numer. Math., vol. 1, pp. 269–271, 1959.

[5] J. Huang, C. Chen, J. Shen, G. Liu, and F. Xu, "A self-adaptive neighborhood search A-star algorithm for mobile robots global path planning," Comput. Electr. Eng., vol. 123, art. 110018, 2025.

[6] H. Guo et al., "Path planning of greenhouse electric crawler tractor based on the improved A* and DWA algorithms," Comput. Electron. Agric., vol. 227, art. 109596, 2024.

[7] W. Wang, J. Li, Z. Bai, and J. Peng, "Toward optimization of AGV path planning: An RRT*-ACO algorithm," IEEE Access, vol. 12, pp. 18387–18399, 2024.

[8] D. Wu and Y. Li, "Mobile robot path planning based on improved smooth A* algorithm and optimized dynamic window approach," in Proc. 2nd Int. Conf. Signal Process. Intell. Comput. (SPIC), Dec. 2024, pp. 345–348.

[9] S. Chen, J. Ji, H. Su, and Y. Yang, "Improved A-star method for collision avoidance and path smoothing," in Proc. IEEE Int. Conf. Control, Electron. Comput. Technol. (ICCECT), May 2023, pp. 32–35.

[10] D. Xu, J. Yang, X. Zhou, and H. Xu, "Hybrid path planning method for USV using bidirectional A* and improved DWA considering the manoeuvrability and COLREGs," Ocean Eng., vol. 298, art. 117210, 2024.

[11] H. Zhang, Y. Tao, and W. Zhu, "Global path planning of unmanned surface vehicle based on improved A-star algorithm," Sensors, vol. 23, art. 6647, 2023.